Convolutional Neural Networks

Ryan Miller

Grinnell College
Statistics

Introduction

At the end of lab 10, you built your own neural network for the dogs/cats image data:

$$64 \times 64 \times 3 \text{ image} \rightarrow \text{Flatten into 12,288 pixels} \rightarrow$$

$$\text{Stack of Linear Layers} \rightarrow \text{Classification}$$

- Flattening was necessary to use `nn.Linear()`
- This network struggled to learn anything, as it ignored the *spatial structure* of the images
  - It could only learn predictive patterns (such as ears, eyes, etc.) if those patterns appeared in the same pixel locations across images

MNIST Example

Here is an image of an 8 in the MNIST handwritten digits dataset:



0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

What features make this an 8? Are those features tied to a certain pixel location? Or could they appear in several different locations?

Convolution

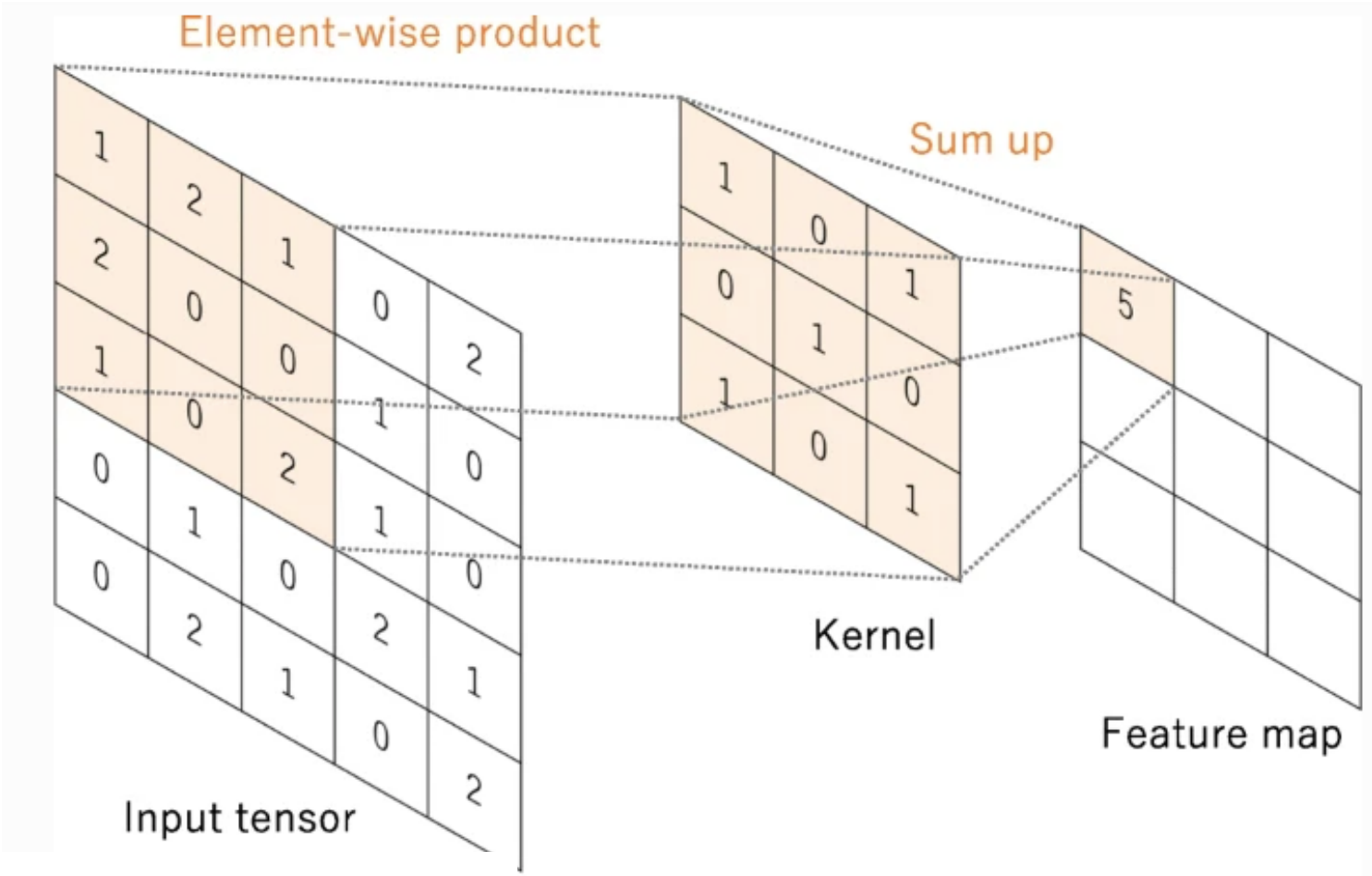
A *convolution layer* learns a small set of weights called a **kernel**:

$$\mathbf{K} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

- Unlike the weights in a fully connected linear layer, these same weights are applied to various locations across the image
  - Thus, patterns can be detected regardless of location, and fewer parameters are needed

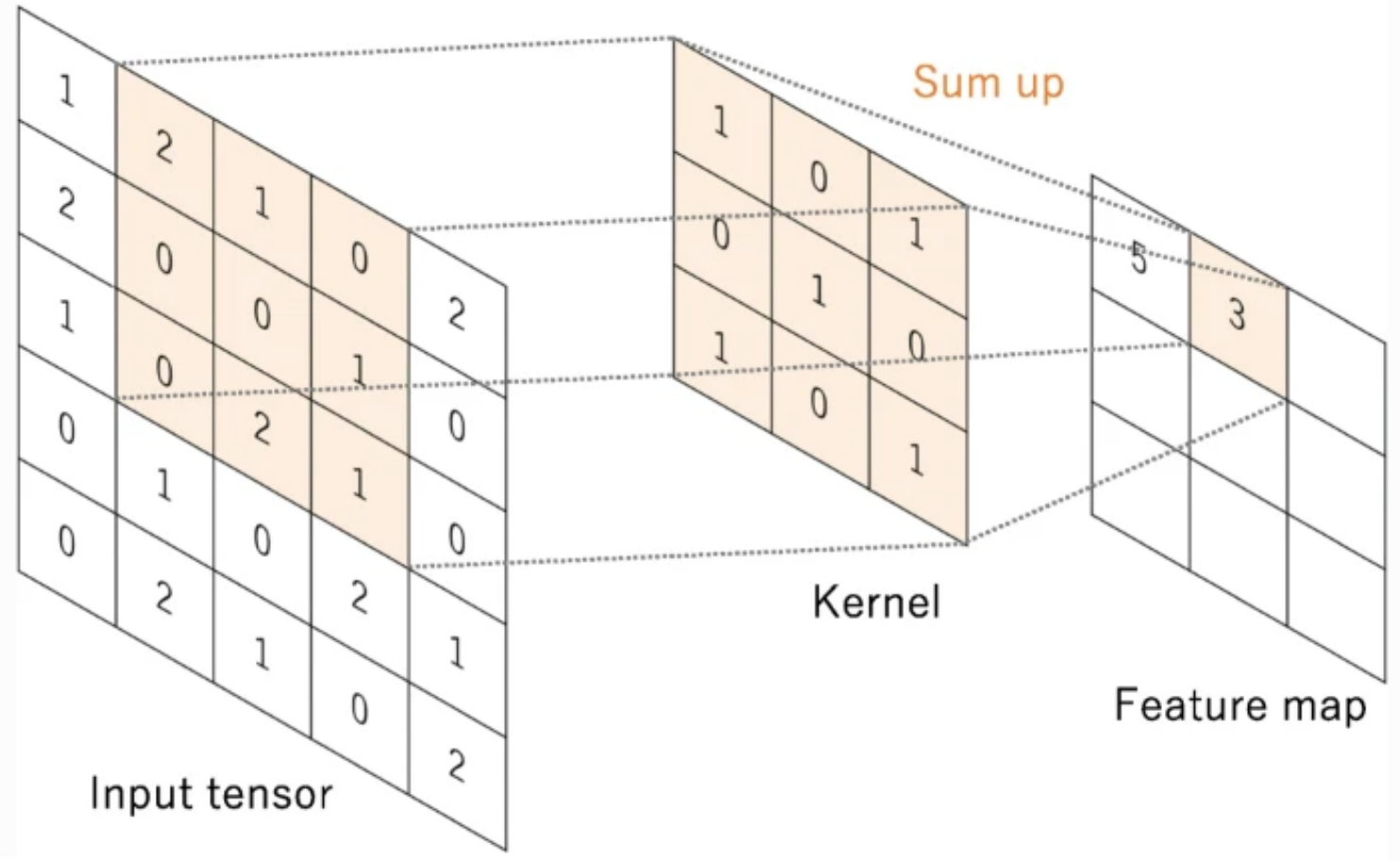
Convolution

Each kernel produces a **feature map**, where each element is the weighted sum of inputs covered by the kernel:



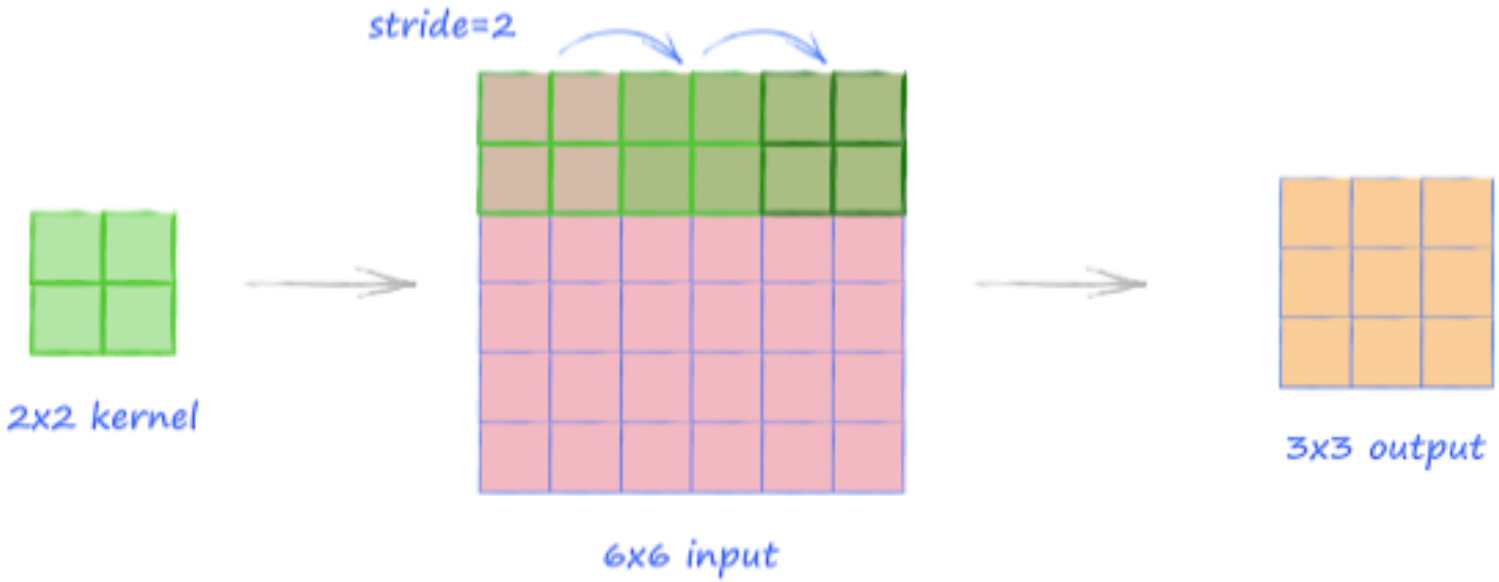
Stride

Stride governs how the kernel is moved across the input to create the feature map. The example below uses a stride of 1:

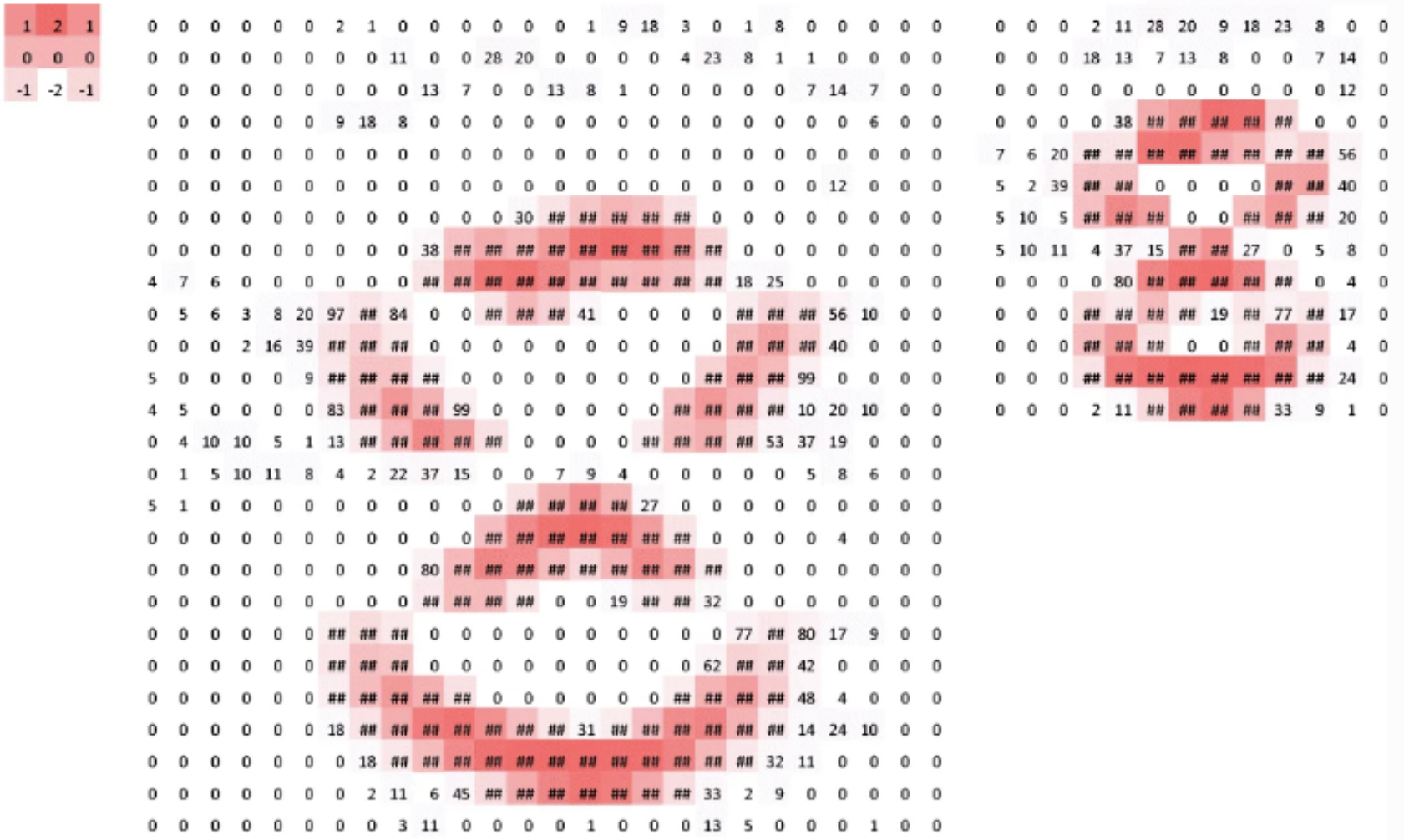


Stride

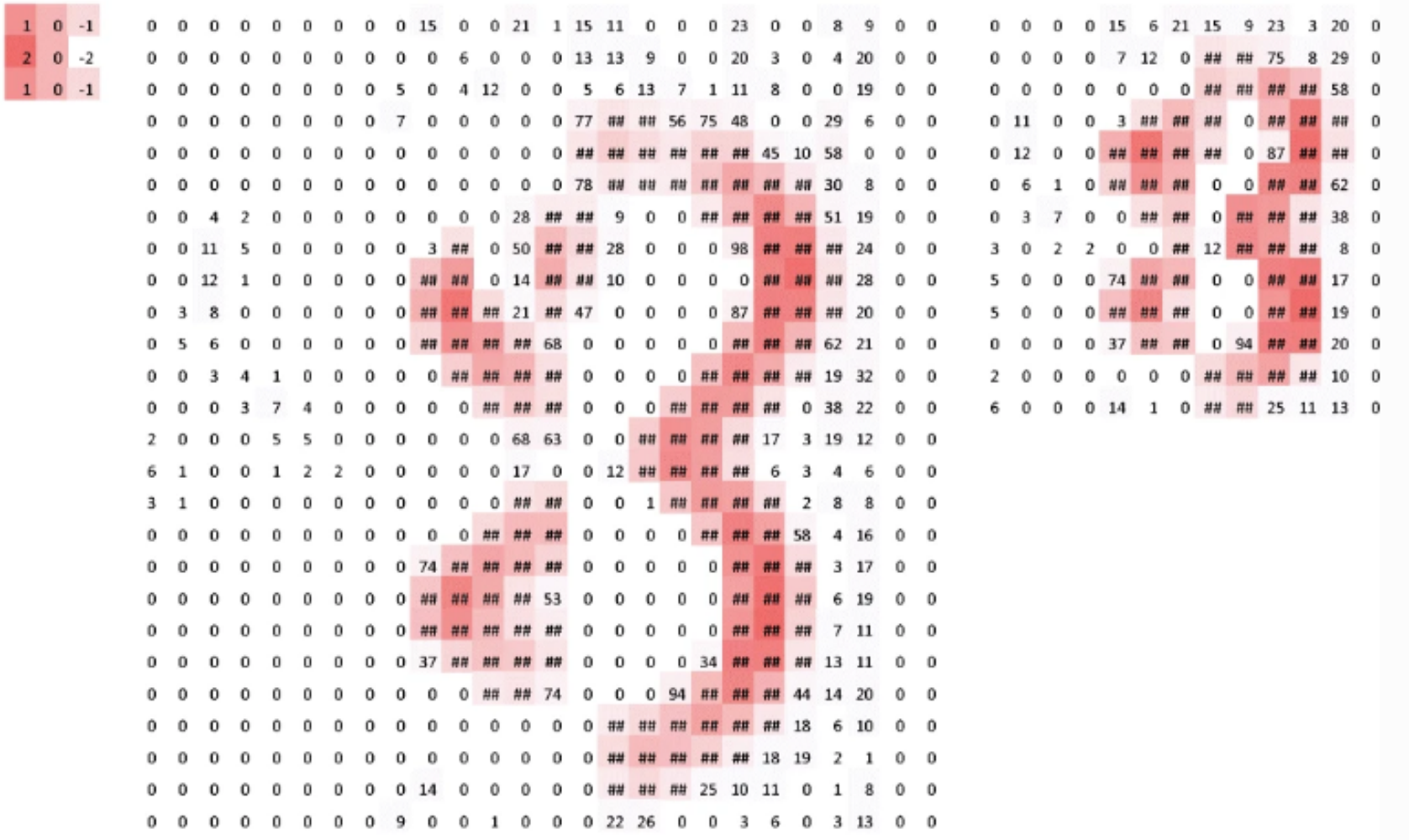
A larger stride will produce a smaller feature map, but might lose information, as not every part of the kernel will see every part of the input:



Example #1 - learning horizontal edges

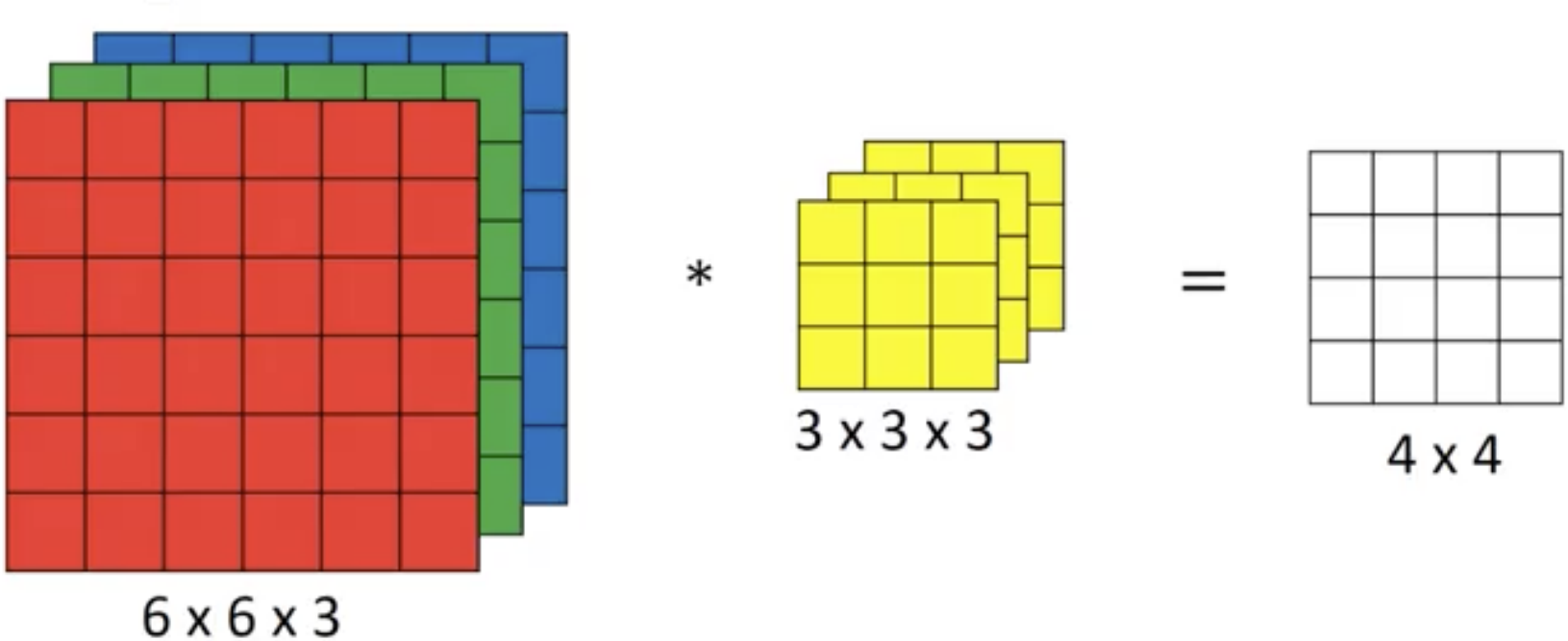


Example #2 - learning vertical edges



Multiple Input Channels

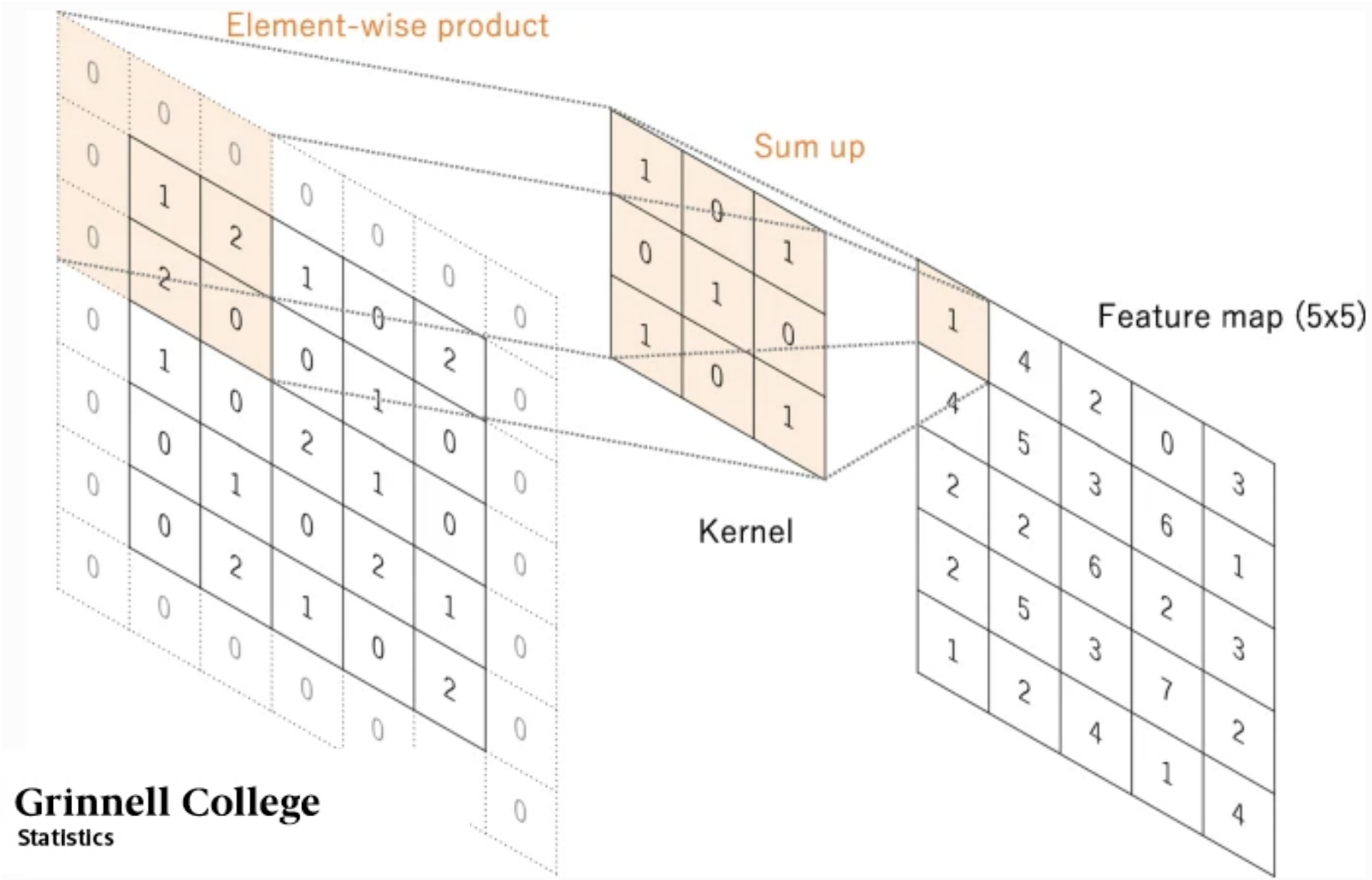
For inputs with multiple channels (such as images with RGB channels), the kernels themselves have multiple channels:



In this example, how many values are summed to produce the (1,1) element in the 4x4 feature map?

Padding

Sometimes patterns of interest appear near the edges of an image. We can ensure the center of a kernel interacts with the edges using **padding**:

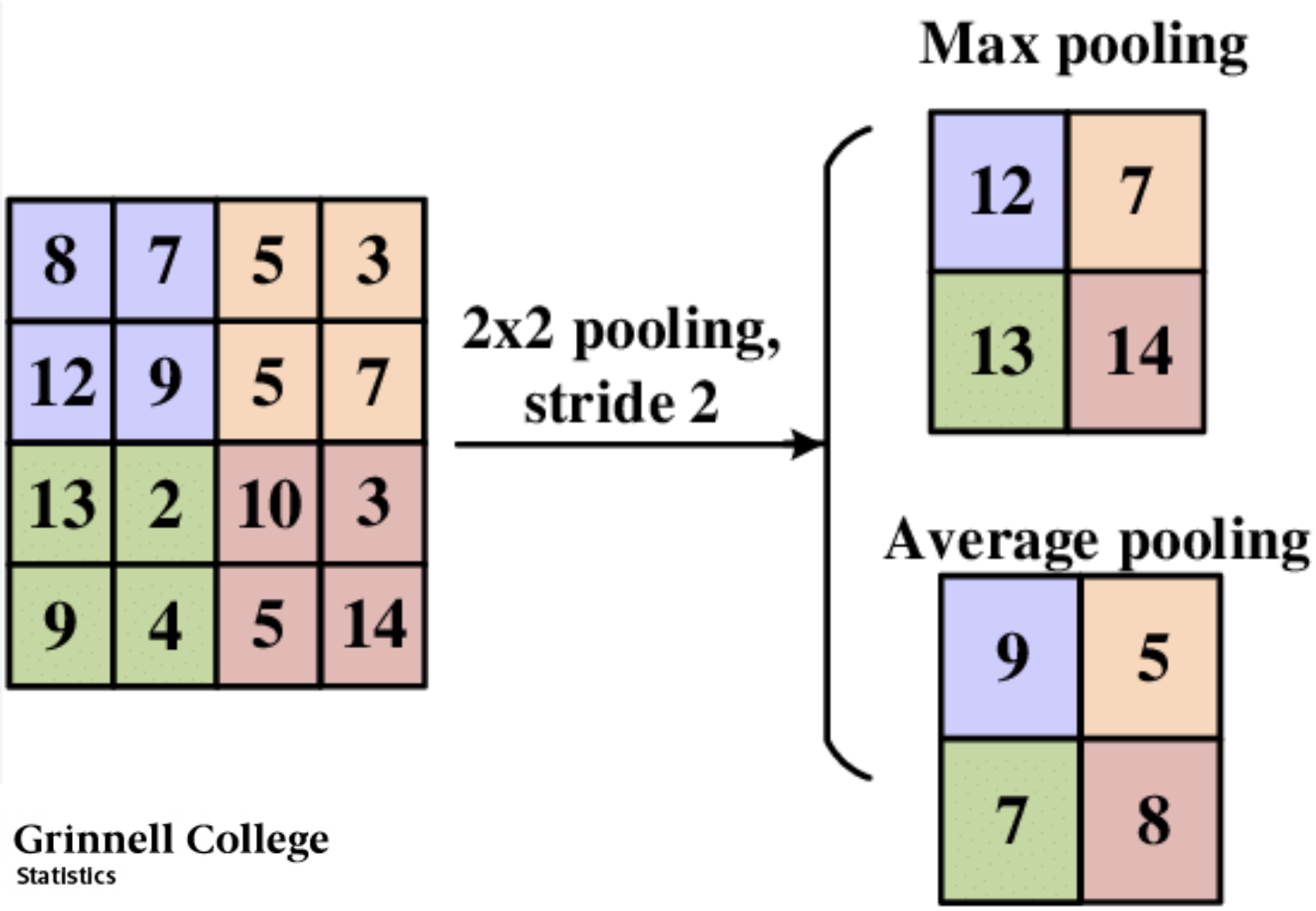


Pooling

- Feature maps can remain large throughout a network, especially when padding preserves their dimensions
- While increasing stride will reduce the size of feature maps, an operation known as **pooling** is generally preferred
  - *Max pooling*, which keeps the maximum value in each "patch" of an input feature map is most commonly used
  - *Average pooling*, which keeps the average value within a "patch" is sometimes used

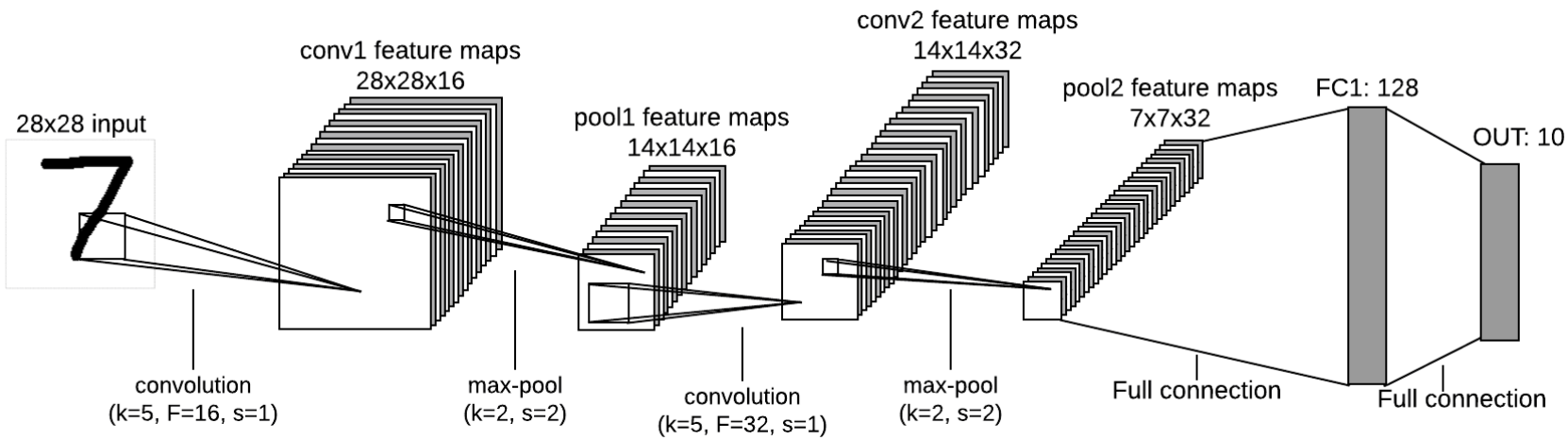
Pooling

The output of a pooling operation depends upon the **patch size** and **stride**. Shown below is pooling using a 2×2 patch with a stride of 2:



Putting it all together

The diagram below demonstrates what the entire architecture of a convolutional neural network might look like:



Comments on Architecture

- Near the end of a convolutional neural network, the feature maps are flattened
  - It's logical to do this when the feature maps no longer contain any meaningful spatial information
- Pooling helps keep the number of parameters in the network under control
  - For example, AlexNet contains roughly 60 million parameters despite using 3 different pooling layers to reduce complexity

Comments on CNNs

- The main benefit of convolution is that kernels are shared across multiple locations of the input
  - Thus, hidden features can be learned from different locations
- In contrast, the hidden features learned in the "vanilla" neural network architectures we've previously discussed are position-sensitive
  - For example, a neuron might learn a horizontal edge, but it could only do so for one specific location in an image
  - If the data were rigorously pre-processed this might be okay, but in general it's a major limitation

Image Credits

Links to images used in this presentation can be found below:

- "Convolutional neural networks: an overview and application in radiology"
- Max pooling vs. Average Pooling
- CNN architecture example